

Breakout

Exploratory Research

Nikola Panayotov
Fontys University of Applied Sciences – ICT

Table of Contents

1 Introduction	3
2 Library research – choosing a technology stack	3
3 Workshop research – exploring the structure before building it	3
4 Lab research – verifying the code does what it should	3
5 Field research – testing with real players	3
6 Showroom research – comparing against existing work	3
7 Conclusion – what triangulating these approaches taught me	4
Sources	4

1 Introduction

LO3 defines exploratory research as: “You explore and ask questions throughout the solution process. You answer questions by using an appropriate approach: pragmatic, critical and based on multiple sources.” This document collects the small research activities carried out during Breakout, following the FHICT triangulation framework (Library, Workshop, Lab, Field, Showroom). Each entry states the question, the approach, the finding, and links to the evidence.

2 Library research – choosing a technology stack

Question: Which language and framework fits Breakout and my own learning goals?

Approach: Compared LibGDX against alternatives using the official documentation and wiki, rather than picking the first option found.

Finding: Java + LibGDX (LWJGL3 backend) was the best fit: well documented, actively maintained, and keeps core Java visible instead of abstracting it away.

Evidence: Breakout Advice, section 3.

3 Workshop research – exploring the structure before building it

Question: Will the planned class structure and screen flow actually hold together?

Approach: Sketched a UML class diagram, screen-flow diagram and wireframes before writing code, so gaps were cheap to catch.

Finding: The screen-flow sketch surfaced a missing state – pausing needed its own screen (PauseScreen) that kept the running game alive rather than recreating it.

Evidence: Breakout Design, sections 2–4.

4 Lab research – verifying the code does what it should

Question: Does the built code actually behave the way the design specified?

Approach: Six controlled checks (T1–T6) against the source, each tied to a specific expected result from the design.

Finding: All six passed: ball physics, paddle flick, armored-brick logic, score timing, pause/resume, and restart all matched the design.

Evidence: Breakout Validation, section 2.

5 Field research – testing with real players

Question: Does the game actually feel good to someone who didn't build it?

Approach: Two peers played the finished build with no instructions; reactions recorded verbatim.

Finding: Both found it polished with no defects against the Must-haves; both independently suggested new features – a timer, sound, and a power-up.

Evidence: Breakout Validation, section 4.

6 Showroom research – comparing against existing work

Question: Are the peer-suggested gaps (sound, power-ups) idiosyncratic, or standard for this genre?

Approach: Compared Breakout against existing open-source Java/Python Breakout-Arkanoid clones on GitHub.

- khoipm06/Arkanoid (Java) – power-ups and sound alongside levels and brick types.
- nello0b/Breakout-Arkanoid (Java) – multiple levels, pause screen, multi-ball.

Finding: Sound and power-ups are standard in comparable clones, not unusual requests. My build already matches or exceeds the norm on levels and multi-hit bricks, but lags on sound and power-ups – independently confirming the peer feedback.

Evidence: GitHub repositories above (see Sources).

7 Conclusion – what triangulating these approaches taught me

Each approach alone gave an incomplete picture: Library and Workshop research made the plan look solid on paper, Lab research confirmed the code matched it, but only Field research showed that “correct” and “enjoyable” aren't the same thing, and only Showroom research told me the gap was a real gap and not just one tester's opinion. Answering the same question from several angles, instead of trusting the first confident answer, is the skill this practiced.

Sources

Technical references consulted:

[khoipm06/Arkanoid – GitHub.](#)

[nello0b/Breakout-Arkanoid – GitHub.](#)